HealthTrack Data Layer and API Design Rationale

Todd Simonson

MSCS 699 Computer Science Capstone

Michael Solomon

November 1, 2026

HealthTrack Data Layer and API Design Rationale

The Phase 2 design converts the Phase 1 requirements into a PostgreSQL schema of 23 tables in seven domains and a REST API of 81 operations documented in OpenAPI 3.1. Three principles drove the major choices: the database enforces clinical rules itself, patient data is visible only to the people caring for that patient, and every table the alert, dashboard, and risk phases need exists now, so those phases add logic instead of restructuring the schema.

Database Design

PostgreSQL was chosen over SQLite or a document store because HealthTrack relies on features they lack. Exclusion constraints stop double-booked clinicians and overlapping exercise sessions, partial unique indexes guarantee one current prescription and one open alert per condition, and row-level security filters rows by caller. Validation is layered by consequence. Impossible values, such as a heart rate of 400, fail a CHECK constraint, while possible but unlikely values are stored with a suspect flag and never alert, because sensor artifacts drive a large share of false alarms (Drew et al., 2014). Cross-column clinical rules also live in the schema: an active home-program patient must be low or moderate risk, matching the eligibility conclusion of Thomas et al. (2019), and only physicians can sign prescriptions or set patient-specific thresholds. Prescriptions and thresholds are versioned rather than overwritten, so every session and alert traces to the limits in effect at that moment. Alert rules, overrides, escalation policies, and risk score bands are rows rather than code, so clinicians can tune them without a release.

API Design

Patient-scoped resources nest under /patients/{patient_id}, which reduces authorization to one scope check on the path, while alerts, sessions, and recommendations have top-level routes because clinicians act on them by identifier. Acknowledging and resolving an alert are explicit action endpoints instead of generic status edits, so each enforces its own preconditions and records a distinct audit action. Device uploads accept batches of up to 500 readings with per-reading idempotency keys, since home connections drop and retries must not duplicate data.

Errors use the RFC 9457 problem details format (Nottingham et al., 2023), and name searches travel in a POST body so patient names never appear in URLs or access logs.

Security

The security design follows the access control, audit control, and transmission security safeguards of the HIPAA Security Rule (U.S. Department of Health and Human Services, n.d.). Access tokens are 15-minute JWTs, refresh tokens are stored only as SHA-256 hashes and rotate on every use, and staff accounts require MFA and lock after five failed logins. Authorization combines role with care-team membership in the API, and the database repeats the membership check through row-level security driven by settings the API sets inside each transaction (PostgreSQL Global Development Group, n.d.). A request without that context returns no rows, so the system fails closed. Administrators see aggregate metrics but no readings, and IT staff see no protected health information. Triggers audit every PHI write by column name without copying values, and the audit log rejects updates and deletes.

Trade-offs and Limitations

These choices have costs. Rules in triggers are harder to test from Python, so the scripts include 58 executable SQL checks that try to break each rule, and Phase 3 tests must run against PostgreSQL rather than SQLite. Row-level security requires every request to run inside a transaction with its context set, which the FastAPI database dependency will handle with SET LOCAL so pooled connections never leak one user's context to another. The vital_signs table will grow fastest; monthly partitioning is deferred to the Phase 6 optimization work, where measured query times can justify it. Finally, all sample data is synthetic, so the plausibility ranges and default thresholds are clinically reasoned starting points that a real program's medical director would need to approve.

References

Drew, B. J., Harris, P., Zègre-Hemsey, J. K., Mammone, T., Schindler, D., Salas-Boni, R., Bai, Y., Tinoco, A., Ding, Q., & Hu, X. (2014). Insights into the problem of alarm fatigue with physiologic monitor devices: A comprehensive observational study of consecutive intensive care unit patients. *PLOS ONE, 9*(10), Article e110274. https://doi.org/10.1371/journal.pone.0110274

Nottingham, M., Wilde, E., & Dalal, S. (2023). *Problem details for HTTP APIs* (RFC 9457). Internet Engineering Task Force. https://doi.org/10.17487/RFC9457

PostgreSQL Global Development Group. (n.d.). *Row security policies*. PostgreSQL documentation. Retrieved October 28, 2026, from https://www.postgresql.org/docs/current/ddl-rowsecurity.html

Thomas, R. J., Beatty, A. L., Beckie, T. M., Brewer, L. C., Brown, T. M., Forman, D. E., Franklin, B. A., Keteyian, S. J., Kitzman, D. W., Regensteiner, J. G., Sanderson, B. K., & Whooley, M. A. (2019). Home-based cardiac rehabilitation: A scientific statement from the American Association of Cardiovascular and Pulmonary Rehabilitation, the American Heart Association, and the American College of Cardiology. *Circulation, 140*(1), e69–e89. https://doi.org/10.1161/CIR.0000000000000663

U.S. Department of Health and Human Services. (n.d.). *Summary of the HIPAA Security Rule*. Retrieved October 28, 2026, from https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/index.html